

A Comparative Empirical Study of FIDO2 and Its Predecessors

Anderko Máté
Óbuda University
mate.anderko@stud.uni-obuda.hu

Abstract—The FIDO2 standard, introduced by the FIDO Alliance and the World Wide Web Consortium (W3C) in 2018, it has been hailed as the “Kingslayer of User Authentication”. This paper presents the implementation of a FIDO2 + WebAuthn authentication system and shows an analysis on different user authentication philosophies. The analysis assesses a FIDO2, a 2FA and a password only system on their resistance to common security threats using the STRIDE threat modeling framework and evaluates performance based on latency, memory usage, and computational overhead. The objective is to determine how adopting FIDO2 influences the security, usability, and system resource requirements of information systems.

Index Terms—FIDO2, WebAuthn, Passkeys, Passwordless authentication, Multi-factor authentication, STRIDE, Performance

I. Introduction

Since the dawn of computing the weakest link in most information systems has been the people creating and using them. We choose easy to remember password that might be prone to dictionary attacks, then we overcomplicate them just to forget them, so to only have to remember a couple passwords we reuse them across sites, also we easily fall for social engineering attacks, like phishing. Sufficiently long, unique, and complex passwords with proper rate limiting remain impractical to brute-force, so compromises overwhelmingly exploit human factors and credential leaks rather than cryptographic weakness. Also the use passwords impose nontrivial support costs and friction because of the resets and account recovery.

To mitigate the weaknesses of a password only environment one-time codes (SMS/TOTP) and push prompts were introduced, this is safety standard is commonly referred to as Multi Factor Authentication MFA materially reduces risk from simple credential stuffing, but common forms remain phishable (e.g., OTP relay, look-alike pages) and are vulnerable to SIM-swap or “push fatigue” attacks. Usability trade-offs (codes, prompts, and device availability) also create drop-off during enrollment and sign-in.

FIDO2/WebAuthn addresses these limitations by replacing shared secrets with origin-bound public-key cryptography and local user verification (biometrics or a device PIN). Private keys never leave the authenticator; the server stores only public keys, eliminating password reuse and greatly reducing the value of stolen databases. Because assertions are tied to the requesting origin, generic

phishing and OTP-relay attacks fail by design. In practice, passkeys (discoverable credentials) further streamline the flow across platforms using platform sync or roaming security keys.

This paper adopts that progression—passwords → MFA → FIDO2—and (i) implements a fully passwordless WebAuthn system, and (ii) compares password-only, password+MFA, and FIDO2 across resilience (phishing, stuffing, brute-force), usability (enrollment, recovery, cross-device), and resource usage (compute, storage, support). We show how MFA mitigates many password risks yet remains phishable, and how FIDO2 offers a principled, practical solution that is both more secure and simpler for end users.

II. Similar and Related Work

This section reviews research on the security, usability, and adoption of FIDO2, also shows a comparison with this here work.

A. Formal and Theoretical Analyses

Barbosa and their team [1] offers a formal model of FIDO2 and, through theoretical analysis, show resistance to credential-phishing and replay attacks. In contrast, the present work examines a deployed FIDO2 stack with STRIDE, emphasizing practical security implications and performance (latency, memory, compute).

B. Empirical and Usability Studies

Lyastani and their team [2] created an empirically FIDO2’s usability analyses. The study shows significant usability gains and phishing resistance compared to passwords and 2FA. Their work relied on user studies and perception metrics rather than system-level measurements.

Unlike these human-centered, behavioral studies, this work isolates the technical performance and security aspects of FIDO2.

C. Positioning of This Work

Key contributions:

- 1) Operational implementation of FIDO2/WebAuthn alongside password-only and 2FA baselines.
- 2) Structured STRIDE analysis, grounded in Barbosa’s theoretical model.

- 3) Empirical performance results—latency, memory footprint, computational load—that complement usability-centric studies (e.g., Lyastani).

Unlike work centered on formal guarantees or user experience, this study examines how adopting FIDO2 shifts real-world security posture and runtime efficiency.

III. Common Authentication Vulnerabilities

Bad actors find the most creative ways to attack authentication systems, these are called attack vectors. The goal of this section is to outline how the vulnerabilities work, which the paper will address in later sections, and also to shine a light on why traditional password or even basic MFA schemes struggle against them.

A. Weak Passwords and Brute-Force Attacks

Users often choose simple passwords, making them easy targets for brute-force attacks [3]. A simple brute-force attack tries to crack the user’s password by trying random character combinations. Rate limiting and sufficient password length makes the attack redundant. A dictionary attack is a more sophisticated version of a brute-force attack, where attackers use common wordlists, lists of frequently used passwords or they systematically guess combinations to find the password. In the case of brute-force not cryptographic weaknesses, but bad password hygiene and the site’s lack of password complexity enforcement leads to the compromise.

B. Credential Stuffing and Password Reuse

Credential stuffing exploits password reuse across websites. In the event of a databreach the user’s password may get leaked, this creates an opportunity for bad actors, who then use the credentials on other sites to try to gain access to other accounts.[4]. The process can be well automated, thus the reuse of credentials turns one breach into many.

C. Social Engineering Attacks

Many attacks rely on human gullability instead of technical flaws. Phishing, SIM swap, and push fatigue attacks are such attacks.

1) Phishing and OTP Relay: Phishing deceives users into revealing credentials on fake websites, that mimic the working of the site the user is familiar with. Attackers can proxy the login and capture OTP codes in real time, bypassing traditional 2FA, this is commonly referred to as a real-time phishing proxy. OTP capturing is Adversary-in-the-Middle type attack[5].

2) SIM Swap Attacks: The way SMS based MFA works is that the user gets an SMS containing a secret code, generated by the server, this while may seems bulletproof requires complete trust in the user’s telecommunication service provider. The attackers may be able to impersonate victims and hijack their phone number (with the service provider unknowingly complicit), thus gaining control of SMS-based OTPs[6].

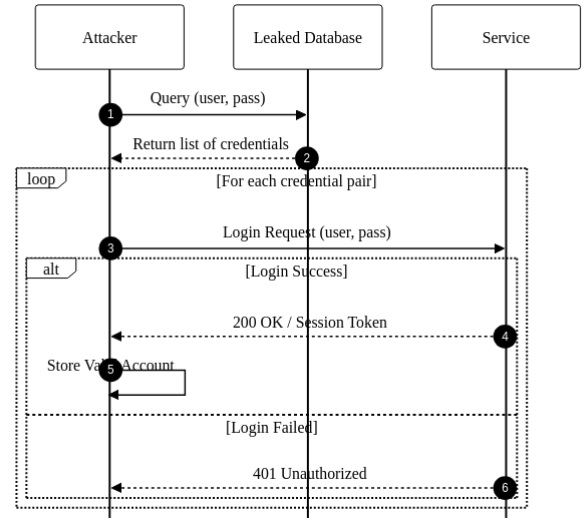


Fig. 1. Credential Stuffing Attack Flow

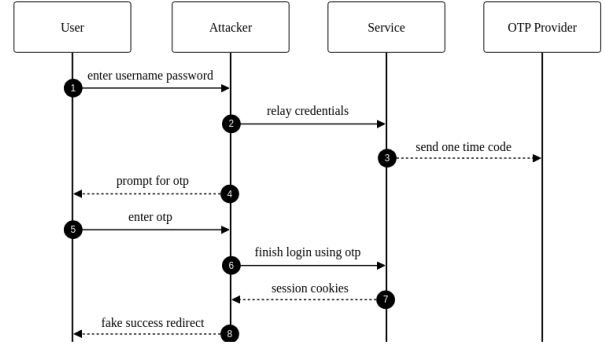


Fig. 2. Phishing with OTP relay

- 3) MFA Push Fatigue: Push-based MFA can be exploited by flooding users with repeated prompts until they approve one, often by mistake or frustration [7].

D. Server-Side Data Breaches

When there is a data breach in the authentication server’s database, password hashes or shared secrets (like TOTP seeds) can get exposed, and then the attackers can crack these sensitive information offline to gain access to the accounts [8].

E. Adversary-in-the-Middle tampering

Interception and modification of traffic between the client and server by a third party falls into the category of tampering. One would say, that TLS provides transport security, but by itself it does not prevent manipulation if it gets compromised or stripped [9].

F. Denial of Service via Account Lockout

Sometimes security measures themselves can be weaponized against the users or the service, attackers

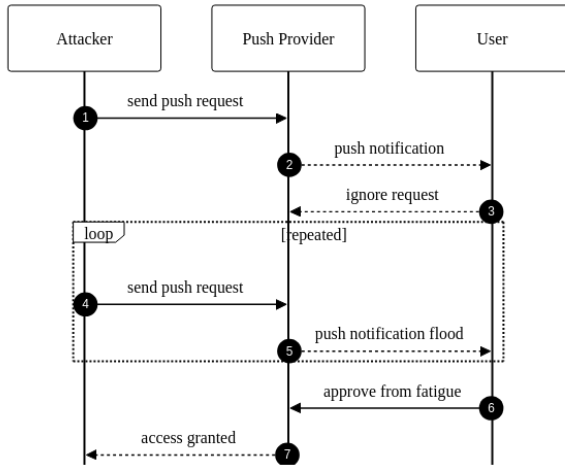


Fig. 3. Push fatigue attack sequence

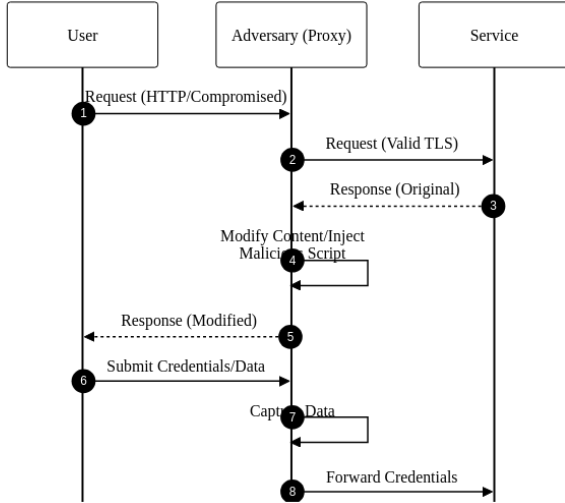


Fig. 4. Replay type AiTM attack flow

trigger account lockouts by repeatedly submitting invalid credentials for a target user. This exploits the lockout policy, while it keeps the user safe from brute-force attacks, it can lead to them not being able to access their account [10].

IV. How FIDO2 Works

This section offers a quick introduction into the FIDO2 ecosystem. FIDO2 provides an phishing-resistant, origin-bound public-key authentication by combining the W3C WebAuthn API (browser/platform) with the FIDO Alliance CTAP protocol.

A. Core Roles

- 1) Relying Party (RP): The application service that issues authentication challenges and stores each user's registered public key.

- 2) Client/Platform: The browser or operating system that brokers WebAuthn requests and responses between the RP and the authenticator.
- 3) Authenticator: A platform or roaming device that creates key pairs, enforces user presence/verification, and returns attested assertions.

B. Registration (makeCredential)

- 1) The RP provides options (challenge, RP ID, user handle, algorithm preferences) to the client.
- 2) The authenticator creates a new key pair based on the RP ID, the private key remains within the authenticator, keeping it safe from attackers, the public key and attestation statement are returned.
- 3) The RP verifies attestation and stores the credential public key.

C. Authentication (getAssertion)

- 1) The RP provides options (challenge, RP ID, and an allow list if applicable).
- 2) The authenticator enforces user presence/verification, then signs the challenge together with the RP ID hash, authenticator flags, and a signature counter.
- 3) The RP verifies origin, RP ID, signature, and counter before completing authentication.

D. Security Properties

- Origin binding (via RP ID and client data) mitigates phishing and relay.
- Non-exportable keys reduce the impact of server-side compromise.
- Attestation enables enforcement of device-class policy, lower class untrusted devices can be blocked.

E. Real-World example

This example tries to convey how Android devices keep the data secure working with the FIDO2 ecosystem. On Android, passkeys and cryptographic keys are managed through Credential Manager and the Android Keystore. Where available, StrongBox places KeyMint in a discrete secure element so that keys are generated and stored within a dedicated secure processor rather than solely in the TEE. Private key material is non-exportable and invocable only via Keystore operations under enforced user-authentication and policy.

- 1) StrongBox isolation: StrongBox uses KeyMint in a separate secure chip. This chip has its own CPU, RAM and secure storage, completely isolating it from the mobile device, thus making it virtually impossible for bad-actors to access the private key stored here. It enables hardware attestation so servers can verify that keys are hardware-backed on a genuine device.

2) Example Flow:

- The registration process begins through the Credential Manager (WebAuthn).
- The Keystore requests a hardware-backed key; on devices with StrongBox, the key pair is generated within the StrongBox KeyMint and marked as non-exportable.
- The public key and attestation chain are returned. The RP may verify the attestation to confirm hardware-backed security.
- During authentication, StrongBox performs the signature after local user verification; the private key never leaves the secure processor.

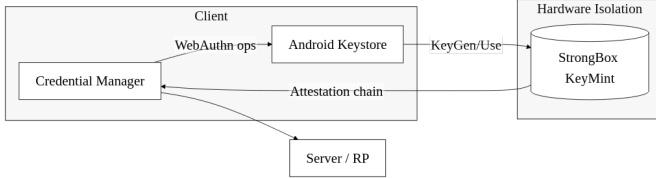


Fig. 5. Android StrongBox flow

V. Implementation

This section talks about the components of, and how they interact in the prototype, that we will run our STRIDE testing and performance analysis on, we will also talk about the packages used in the backend, this is important to reflect on in when evaluating the results.

A. Backend Packages

We use the backend packages, listed in Table I. Python’s FastAPI framework handles requests, while ‘pydantic’ ensures data integrity. For our database we use PostgreSQL (via ‘psycopg2-binary’) as it is FOSS and is widely used in the industry, to link our database to our application we use ‘sqlalchemy’, which serves as an ORM layer, so we can interact with the database in a type-safe way. The package ‘redis’ stores state objects, like WebAuthn challenges. Session management is done by ‘pyjwt’ package, it has the job of issuing JSON Web Tokens upon successful login.

We implement different authentication methods:

- Password: For hashing the user’s password we use the ‘bcrypt’ library, configured with a work factor of 12 salt rounds to align with OWASP recommendations [8], according to the guidelines, this should ensure that stored credentials remain resistant to rainbow table and brute-force attacks while maintaining acceptable server performance (200ms).
- 2FA (TOTP): Besides the password management we also had to implement a 2FA system, using the Time-based One-Time Password algorithm. We chose the ‘pyotp’ library, supported by ‘qrcode’ for provisioning,

we follow RFC 6238 [11] during the implementation, we used HMAC-SHA1 based OTP, requiring a 6 digit code.

- FIDO2/WebAuthn: For credential registration and authentication we use the official ‘fido2’ library with ‘cbor2’ for binary data decoding.

TABLE I
Backend Python packages and their roles

Package	Role
‘fastapi’	Web framework providing REST API
‘uvicorn’	ASGI server, runs FastAPI
‘fido2’	FIDO2 credential handling
‘sqlalchemy’	ORM to access DB
‘psycopg2’	PostgreSQL database adapter
‘pydantic’	Data validation management
‘redis’	State store for challenge data
‘pyjwt’	JWT token generation and verification
‘cbor2’	CBOR encoding/decoding binary data
‘bcrypt’	Password hashing
‘pyotp’	TOTP code generation and verification
‘qrcode’	QR code generation for TOTP

VI. STRIDE Threat Modelling Framework

When evaluating the security of the authentication methods, we applied a framework that allow us to analyse them and compare their security, the tool of our choosing was the STRIDE threat modeling framework. It creates an abstraction of real-world attacks by grouping them into the following types:

STRIDE categories:

- Spoofing: In a spoofing attack the bad actor pretends to be a valid user to gain access, this may happen via credential theft or session hijacking.
- Tampering: Tampering attacks mainly focus changing on the message content in transit, while staying undetected.
- Repudiation: Repudiation can be defined by successfully stating that an action was not performed by the user.
- Information Disclosure: Information disclosure is the category, where sensitive data gets exposed to actors with no permission to access it.
- Denial of Service: In a DoS attack one or more user loses access to the service. This may happen by flooding the server with requests or by getting users locked out of their accounts.
- Elevation of Privilege: Elevation of privilege happens when bad actors get admin rights. This may happen by stealing credentials or adding themselves to more privileged groups.

VII. STRIDE Analysis

This section aims to further explain the threat mapping presented in Table ??.

A. Spoofing

Password-only: Password-only systems are the most vulnerable to phishing attacks and credential stuffing, since they entirely rely on the user's better judgement, there is no origin binding, or relying on a safe third party. Expectation: The system should accept valid credentials regardless of their source, making it vulnerable to credential stuffing from leaked databases.

2FA: 2FA creates a much stronger barrier against phishing attacks, since it provides another wall of defence, in the form of a second factor. Depending on the type of this factor attackers can use different ways to bypass it mostly via Real Time Phishing (Relay Attacks), or Sim Swapping. Expectation: The system should accept a valid TOTP code even if it is relayed immediately, provided it is within the validity window.

FIDO2/WebAuthn: FIDO2's origin binding ensures that even if an attacker manages to phish the user, they can't redirect the user to a fake site, since the system would not accept the authentication request from a foreign RP ID. Expectation: The system should reject authentication attempts where the relying party ID does not match the server's origin, effectively preventing phishing.

B. Tampering

Password-only: Usually password only systems offer no protection against message tampering beyond TLS. If TLS is compromised or misconfigured, an attacker can modify login requests or responses. Expectation: Without transport layer security, the system should be unable to detect if the credentials have been modified in transit.

2FA: 2FA TOTP systems also rely primarily on TLS for tampering protection. OTP codes can be intercepted during transmission, for example in a relay attack where the attacker uses a proxy to phish the user's password and intercept the OTP code, then uses it to authenticate. Expectation: Similar to password systems, the integrity of the OTP code relies entirely on the transport layer.

FIDO2/WebAuthn: Since FIDO2 is a digitally signed challenge-response system, modification to the challenge or response invalidates the signature, making it virtually tamper-proof even if TLS is compromised. Expectation: The system should reject any authentication attempt where the payload has been modified, as the digital signature will fail verification.

C. Repudiation

Password-only: There is no repudiation protection in a barebones password system, anyone with the password can authenticate and there is no way to link a particular login to a device or authenticator. Even logging can be manipulated since we can not be sure about the user's identity. Expectation: The system should not be able to provide cryptographic proof of who performed an action, as the server possesses the shared secret.

2FA: 2FA can provide some repudiation protection by logging OTP usage, but these logs can be modified by attackers who manage to gain high enough privileges. Proving the who performed the authentication is difficult in these systems, since cryptographic proof of which device generated the OTP code is not provided by the standard. Expectation: The system should lack non-repudiation properties since the shared secret is stored on the server and logs can be manipulated.

FIDO2/WebAuthn: Since the private keys are stored on the authenticator, and these encapsulated systems manage them FIDO2 offers strong repudiation protection, because other devices can not copy this information. The signature counter also helps detect credential cloning attempts. Expectation: The system should provide strong non-repudiation, as the server stores a digital signature that can only be generated by the user's private key.

D. Information Disclosure

Password-only: Password systems are highly vulnerable to information disclosure. Passwords are stored as hashes, but breaches can expose these hashes for offline cracking. More critically, passwords are transmitted during every login, creating multiple opportunities for interception. Password reuse across sites amplifies the impact of any single breach. Expectation: A database breach should reveal password hashes that are vulnerable to offline cracking attacks.

2FA: TOTP reduces information disclosure risk compared to passwords. The TOTP secret is stored server-side and never transmitted, but if compromised, it can generate valid codes. OTP codes themselves are short-lived but can be intercepted during transmission. Backup codes, if stored improperly, can also be a disclosure risk. Expectation: A database breach should reveal the shared TOTP secrets, allowing an attacker to generate valid codes.

FIDO2/WebAuthn: One of the main principles of FIDO2 is that shared secrets are transmitted only public keys are stored server-side, and private keys never leave the authenticator. Even in an event of databreach the attacker can only get the public keys, which are not enough to authenticate. Expectation: A database breach should only reveal public keys, which are insufficient for an attacker to authenticate as a user.

E. Denial of Service

Password-only: There are multiple DoS attack vectors for password systems, and bad actor might use brute force, to overwhelm the server, but there are certain types that focus on attacking a single user via the server's security features like lockout mechanisms. An attacker might trigger lockouts by repeatedly attempting incorrect passwords, preventing legitimate users from accessing their accounts. Authentication systems usually implement countermeasures, like rate limiting, but these are not

always effective. Expectation: The system should lock out a user account after a defined number of failed attempts, creating a vector for targeted denial of service.

2FA: 2FA systems inherit the vulnerability of password-only systems, the lockout mechanism may shift to the OTP code level, but the attackers can also exhaust backup codes, flood users with OTP requests. Expectation: The system should exhibit similar lockout behavior to password systems when subjected to multiple invalid OTP attempts.

FIDO2/WebAuthn: Passwordless authentication is resilient to DoS attacks, its biggest vulnerability is the hardware doing the authentication, like wiping data from the authenticator in a CTRAPS attack, which factory resets private keys. On the server side there is no incentive to penalise user with lockouts, since there is no password for the attackers to guess. Expectation: The system should handle invalid authentication attempts without locking the user account, as there is no secret to protect from guessing.

F. Elevation of Privilege

Password-only: Since administrative accounts without the proper precaution can fall victim to a multitude of attacks, bad actors can gain initial access to administrative accounts and elevate their privileges. Expectation: Gaining access to an administrator's credentials should allow an attacker to fully take over the administrative account.

2FA: 2FA and password only systems are similarly vulnerable to privilege elevation attacks, even though the additional factor makes them more difficult, with the proper safety measures in place seemingly impossible for attackers to gain access, the elevation of privilege is still possible with the aforementioned techniques. Expectation: Phishing an administrator's password and OTP code should allow an attacker to elevate privileges.

FIDO2/WebAuthn: For FIDO2 elevation of privilege attacks do not usually target the cryptographic protocol, but they may try to register their FIDO keys to a high privilege account, granting them access to rights they should not have. Expectation: Knowledge of an administrator's username should not allow privilege escalation without physical possession of the registered authenticator.

VIII. Empirical STRIDE Security Testing

This section presents the empirical validation of the STRIDE threat analysis described in Section VII. To verify the theoretical security properties of the three authentication methods—password-only, TOTP-based 2FA, and FIDO2/WebAuthn—we developed an automated security testing suite. The objective was to confirm the presence of expected vulnerabilities in legacy systems and validate the security protections provided by FIDO2.

A. Test Methodology

The security tests were implemented in the pytest framework. To ensure the results are reliable and reproducible, the testing followed the key points of:

- **Environment Isolation:** Each test runs in a separate, clean environment. We used in-memory databases for every test case. This prevents interference between tests and ensures consistent results.
- **Full System Testing:** The test - backend interaction is achieved by using an asynchronous client (httpx) in an end-to-end manner. This simulates how a bad actor would interact with the system.
- **Hardware Simulation:** For FIDO2 tests, we created a MockAuthenticator software component, that was used for cryptographic signing and key management, bypassing the requirement for a WebAuthn authenticator Hardware, allowing us to generate both valid and invalid authentication data to test the server's response to various attack scenarios.

1) Spoofing:

- **Password-only:** Impersonation using only a password was straightforward via a credential stuffing attack. When the test case used a simulated "leaked" password from a different site, the server accepted the attempt. This confirmed that the password was not origin-bound, making the account vulnerable if credentials are reused.
- **2FA:** To test TOTP, we executed a relay attack. By generating a valid TOTP code and immediately passing it to the server, we simulated an attacker intercepting the code in real-time. The server accepted the code, showing that TOTP does not inherently prevent relay attacks.
- **FIDO2/WebAuthn:** We simulated a phishing attempt by using MockAuthenticator to log in with an incorrect RP ID. This mimics a user trying to log in on a fake website. The server correctly rejected the login because the cryptographic signature did not match our expected origin, proving its resistance to phishing.

2) Tampering:

- **Password only and 2FA:** Since password-only and 2FA systems usually entirely rely on TLS (or even the old SSL) for integrity, in our tests we were able to modify the user's credentials to that of an attacker, without the server detecting it (In the simulated case where there is no TLS layer. This can be done by attacks like SSL stripping)
- **FIDO2/WebAuthn:** The FIDO2 suite uses digital signatures, when we tried modifying it along the data payload in a valid FIDO2 response, the server rejected every such request. This confirmed that FIDO2 systems are able to verify the integrity of the data, independent of the network layer.

3) Repudiation: Repudiation involves a user denying they performed an action. We tested the system’s ability to provide proof of identity.

- Password only and 2FA: The tests showed that password and TOTP logs are not completely secure, the server knows the secrets, database administrators could forge log entries. There is no proof that ties an action to a specific user.
- FIDO2/WebAuthn: A digital signature consists of hash encrypted with the user’s unique private key, the server stores this signature, which provides strong evidence that the specific user authorized the action. Our test confirmed that the server can verify the signature, and that it is unique to the user.

4) Information Disclosure: Information disclosure when sensitive data gets exposed. We simulated a database breach to see what an attacker would gain.

- Password only and 2FA: For password systems, the test retrieved password hashes, which can be cracked offline. In the case of the 2FA TOTP system, the test retrieved the shared secrets, allowing an attacker to generate valid codes.
- FIDO2/WebAuthn: For FIDO2, the test confirmed that the database contains only public keys. We verified that these keys cannot be used to authenticate, meaning a database breach does not compromise user accounts.

5) Denial of Service (DoS): In a DoS attack, the goal is to make the system unusable, either on a global scale or for a specific user. We tested how rate limiting can be used in these types of attacks.

- Password only and 2FA: We sent many invalid password attempts to a user account. The system locked the account to prevent guessing. This confirmed that the security mechanism itself can be used to deny access to legitimate users. This could be considered a fair tradeoff, as it prevents brute force attacks.
- FIDO2/WebAuthn: We sent invalid requests to the FIDO2 endpoint. The server rejected the invalid signatures efficiently. Because there is no shared secret to guess, the system does not need to lock accounts as aggressively, making it more resilient to this type of attack.

6) Elevation of Privilege: Elevation of privilege involves gaining higher access rights. We tested the security of administrative accounts.

- Password only and 2FA: We simulated a phishing attack against an administrator. For password and TOTP systems, obtaining the credentials allowed full access to the admin account.
- FIDO2/WebAuthn: For FIDO2, we showed that knowing the admin’s username was not enough. The test confirmed that authentication failed without the physical security key, proving that remote attackers

cannot elevate privileges even if they have user information.

IX. Performance and Resource Analysis

This section presents an empirical evaluation of the performance and resource consumption of the implemented authentication methods. The analysis focuses on end-to-end latency, server-side CPU overhead, and database load, providing a comparative assessment of Password-only, TOTP-based 2FA, and FIDO2/WebAuthn systems.

A. Latency Analysis

We measured the end-to-end latency for registration and login operations to evaluate the user experience. The tests were conducted in an isolated environment to minimize external network variance.

1) Registration Performance: Figure 6 illustrates the registration latency for each method.

Password Registration (≈ 216 ms): The high latency is intentional. We use the bcrypt hashing algorithm. It is designed to be slow to resist brute-force attacks [3]. This consumes significant server time.

TOTP Setup (≈ 228 ms): This method inherits the latency of password registration. It adds a small overhead for secret generation. The QR code generation adds negligible time. The bcrypt step remains the bottleneck.

FIDO2 Registration (≈ 108 ms): This is the fastest server-side method. The server only verifies a signature. It does not perform expensive hashing. The complex key generation happens on the client device [2].

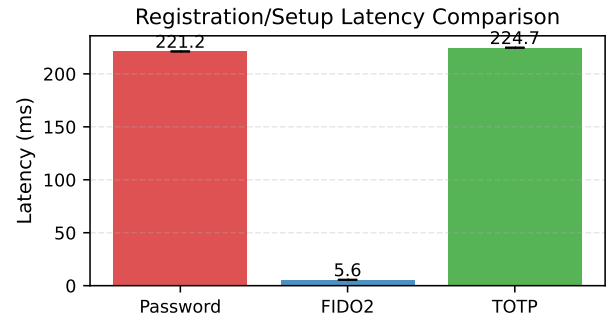


Fig. 6. Registration latency comparison. FIDO2 outperforms legacy methods by offloading cryptography to the client.

2) Login Performance: Login latency results are shown in Figure 7.

Password Login (≈ 216 ms): The server must re-hash the input password. This takes the same amount of time as registration. It is the dominant cost factor.

TOTP Login (≈ 216 ms): This is statistically identical to password login. The HMAC-SHA1 check for the code is extremely fast. It takes fractions of a millisecond. It adds no perceptible delay [5].

FIDO2 Login (≈ 102 ms): The server performs an ECDSA signature verification. This is mathematically efficient [1]. It is much faster than bcrypt hashing.

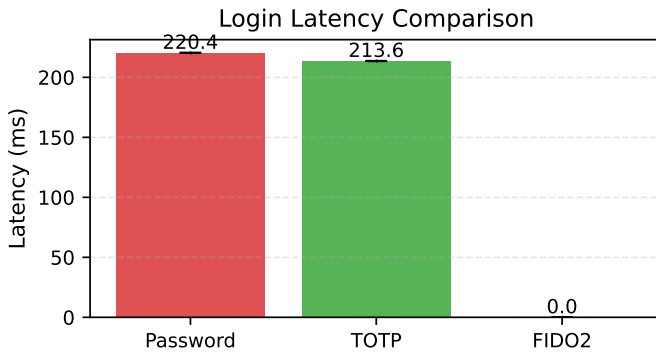


Fig. 7. Login latency comparison. Password and TOTP latencies are nearly identical due to the dominance of bcrypt hashing.

It is important to note that our server-side measurements for FIDO2 exclude the authenticator hardware processing time. Research indicates this introduces a delay of 50–200 ms depending on the device [12]. Even with this delay, FIDO2 remains competitive while offering superior security.

B. Resource Usage

1) Computational Overhead: Figure 8 compares the server-side CPU time.

Password/TOTP: These methods are CPU intensive. Every login burns server cycles on bcrypt. This limits scalability under high load.

FIDO2: This method is CPU efficient. The heavy cryptography is offloaded to the user’s device. The server only performs lightweight verification. This allows for higher throughput [2].

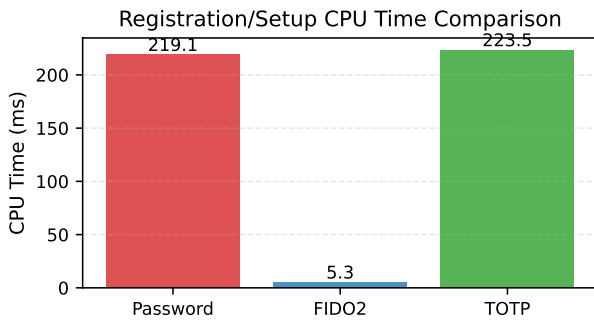


Fig. 8. Server-side CPU time comparison. FIDO2 significantly reduces server load by eliminating server-side hashing.

2) Database Load: Database query analysis is shown in Figure 9.

Registration: Password and TOTP flows often require multiple steps. This results in more database queries. FIDO2 registration is streamlined. It requires fewer round-trips to the database.

Login: All methods are highly optimized. They typically require only a single lookup. Database load is not a differentiating factor for login.

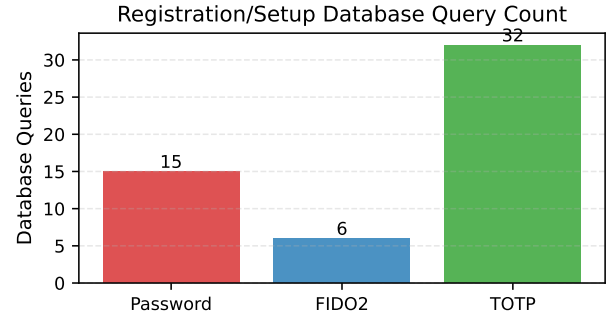


Fig. 9. Database query count comparison for registration flows.

References

- [1] M. Barbosa et al., “Provable security analysis of fido2,” in Information Security Theory and Practice (WISTP 2021). Springer, 2021. [Online]. Available: https://dl.acm.org/doi/10.1007/978-3-030-84252-9_5
- [2] S. Lyastani et al., “Is fido2 the kingslayer of user authentication?” in IEEE Symposium on Security and Privacy, 2020. [Online]. Available: <https://trust.cispa.saarland/publication/lyastani-20-sp/lyastani-20-sp.pdf>
- [3] E. Ma, S. Hasama, E. Lumba, and E. Birrell, “Prospects for improving password selection,” arXiv preprint arXiv:2201.01350, 2022.
- [4] A. Nisenoff, M. Golla, M. Wei, J. Hainline, H. Szymanek, A. Braun, A. Hildebrandt, B. Christensen, D. Langenberg, and B. Ur, “A two-decade retrospective analysis of a university’s vulnerability to attacks exploiting reused passwords,” in Proc. USENIX Security Symposium, 2023.
- [5] M. Käsemodel, V. Winterhalter, F. Heisel, F. Alt, and B. Pfleging, “BlueTOTP: Designing phishing-resistant and user-friendly two-factor authentication,” in Proc. IFIP INTERACT 2025 (LNCS 16110). Springer, 2025, pp. 611–634.
- [6] K. Lee, B. Kaiser, J. Mayer, and A. Narayanan, “An empirical study of wireless carrier authentication for SIM swaps,” in Proc. 16th Symposium on Usable Privacy and Security (SOUPS). USENIX Association, 2020, pp. 61–79.
- [7] Cybersecurity & Infrastructure Security Agency (CISA), “Implementing phishing-resistant mfa (fact sheet),” <https://www.cisa.gov/sites/default/files/publications/fact-sheet-implementing-phishing-resistant-mfa.pdf>, 2022, accessed 2025-10-29.
- [8] OWASP Foundation, “Password storage cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2024, accessed: 2025-11-26.
- [9] —, “Manipulator-in-the-middle attack,” https://owasp.org/www-community/attacks/Manipulator-in-the-middle_attack, 2024, accessed: 2025-11-26.
- [10] National Institute of Standards and Technology, “Digital identity guidelines: Authentication and lifecycle management (nist special publication 800-63b),” U.S. Department of Commerce, Tech. Rep., 2017, includes updates as of 2024.
- [11] D. M’Raihi, J. Rydell, F. Hoornaert, S. Machani, and P.-C. Hsiang, “TOTP: Time-Based One-Time Password Algorithm,” RFC 6238, May 2011. [Online]. Available: <https://www.rfc-editor.org/info/rfc6238>
- [12] L. Würsching, S. Putz, M. Häslar, and M. Hollick, “Platform vs. roaming authentication on smartphones,” in CHI Conference on Human Factors in Computing Systems, 2023. [Online]. Available: https://peasec.de/paper/2023/2023_WuerschingPutzHaeslerHollick_PlatformvsRoamingAuthenticationonSmartphones_CHI.pdf